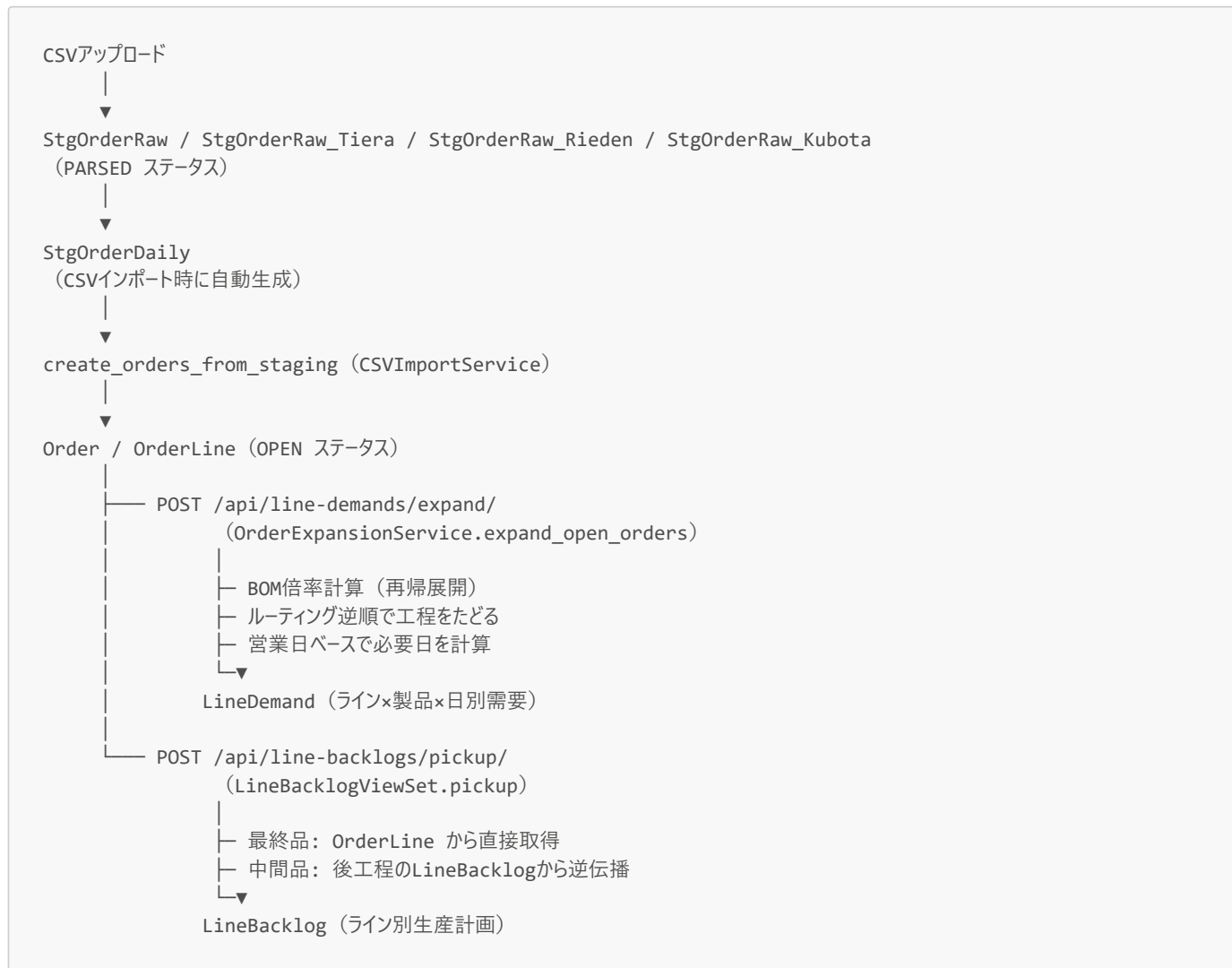


受注展開仕様書

受注データ（Order/OrderLine）からライン別需要（LineDemand・LineBacklog）への展開ロジックをまとめる。

1. 全体フロー



2. ステージング → Order/OrderLine 生成

実装

- **ファイル:** `pm_backend/apps/orders/core/services/csv_import.py`
- **クラス:** `CSVImportService`
- **メソッド:** `create_orders_from_staging()`

対象レコード

ステータスが **PARSED** の StgOrderDaily のみを対象とする。

```
query = StgOrderDaily.objects.filter(
    Q(raw__parse_status='PARSED') |
    Q(raw_kubota__parse_status='PARSED') |
    Q(raw_tiera__parse_status='PARSED') |
    Q(raw_rieden__parse_status='PARSED')
)
```

グループ化単位

(`customer_id`, `order_type`, `version_no`, `source_file`) の組み合わせで1つの Order ヘッダを生成する。

order_no 生成ルール

受注タイプ	顧客	フォーマット
FORECAST	Kubota	FC- <code>{customer_code}</code> - <code>{calc_date}</code>
FORECAST	その他	FC- <code>{customer_code}</code> - <code>{timestamp}</code>
FIRM	Kubota (特殊)	FIRM- <code>{customer_code}</code> - <code>{factory}</code> -sp- <code>{issue_date}</code>
FIRM	Kubota (通常)	FIRM- <code>{customer_code}</code> - <code>{factory}</code> - <code>{issue_date}</code>
FIRM	Rieden	FIRM- <code>{customer_code}</code> - <code>{order_date}</code>
FIRM	Tiera	FIRM- <code>{customer_code}</code> - <code>{delivery_no}</code>
FIRM	その他	FIRM- <code>{customer_code}</code> - <code>{timestamp}</code>

置換戦略

受注タイプ	戦略
FORECAST (内示)	部分置換：新データの最小納期以降の同製品の既存ラインを SUPERSEDED に変更し、新規作成
FIRM (確定)	原則新規追加。Special Kubota (<code>is_kubota_special=True</code>) のみ既存削除→新規作成

3. 受注展開 → LineDemand 生成

実装

- **ファイル**: `pm_backend/apps/production/services/order_expansion.py`
- **クラス**: `OrderExpansionService`
- **メソッド**: `expand_open_orders(clear_existing: bool = True)`
- **API**: `POST /api/line-demands/expand/`

処理ステップ

Step 1: 対象受注の取得

```
order_lines = OrderLine.objects.filter(
    order__status='OPEN'
).select_related('order', 'product')
```

ステータス **OPEN** のみ対象。 **SUPERSEDED** は除外。

Step 2: BOM 倍率計算 (`_get_bom_multiplier_maps`)

BOM ツリーを再帰展開し、最終品1個あたりの各中間品の所要量を計算する。

```
最終品A (1個)
├─ 部品B (2個)
│   └─ 部品C (1個)
└─ 部品D (3個)

bom_multiplier = {A:1, B:2, C:2, D:3}
path_multiplier = {"1":2, "1.1":2, "2":3}
```

連産品BOM (is_coproduct=True) は除外する。

Step 3: ルーティング按分との補正

ルーティング上の工程倍率と BOM 倍率が異なる場合に補正係数を計算する。

```
correction = bom_multiplier[pid] / routed_multiplier[pid]
```

Step 4: 必要日の計算（営業日ベース）

ルーティングの `hierarchy_path` を親→子の順でたどり、工程ごとの必要日を計算する。

```
最終工程の必要日 = 受注納期 - リードタイム（営業日）
前工程の必要日   = 後工程の必要日 - 前工程のリードタイム（営業日）
```

リードタイムの優先順位：

1. `RoutingStep.lead_time_days`（工程固有）
2. `Line.lead_time_days`（ライン共通）
3. 0（デフォルト）

Step 5: ライン別に数量を集計

```
# キー: (line_id, product_code, plan_date)
for step in reversed(steps): # 後工程から前工程へ逆順処理
    qty = order_line.quantity * base_mult * correction
    if order_type == 'FIRM':
        aggregated[key].firm_qty += qty
    else:
        aggregated[key].forecast_qty += qty
```

Step 6: LineDemand レコードを一括保存

```
with transaction.atomic():
    if clear_existing:
        LineDemand.objects.all().delete() # 全件削除・再作成
        LineDemand.objects.bulk_create(objects_to_create)
```

`clear_existing=True`（デフォルト）の場合、**既存のLineDemand全件を削除**して再作成する（フル再計算）。

LineDemand の主要フィールド

フィールド	内容
<code>line_id</code>	ライン
<code>routing_step_id</code>	ルーティング工程
<code>product_id</code> / <code>product_code</code>	製品
<code>plan_date</code>	必要日（納期 - リードタイム）
<code>lead_time_days</code>	リードタイム（日）

フィールド	内容
forecast_qty	内示数量
firm_qty	確定数量
plan_qty	計画数量（初期値 = forecast + firm）
actual_qty	実績数量（初期値 0）
order_numbers	紐付き受注番号（カンマ区切り、500字まで）

4. LineBacklog 生成（pickup）

実装

- **ファイル**: pm_backend/apps/production/views.py
- **クラス**: LineBacklogViewSet
- **アクション**: POST /api/line-backlogs/pickup/

入力パラメータ

```
{
  "line_id": 1,
  "start_date": "2025-01-01",
  "end_date": "2025-01-31"
}
```

最終品の需要取得

最終品（`is_final_product=True`）は **OrderLine** から直接取得する。

```
order_lines = OrderLine.objects.filter(
    order__status='OPEN',
    product_id__in=final_products
)
for ol in order_lines:
    lead_days = resolve_lead_time_days(ol.product_id)
    plan_date = shift_business_days(ol.due_date, lead_days)
    if ol.order.order_type == 'FIRM':
        firm_map[(ol.product_id, plan_date)] += ol.quantity
    else:
        forecast_map[(ol.product_id, plan_date)] += ol.quantity
```

中間品の需要取得（後ライン逆伝播）

中間品は後工程の **LineBacklog** から BOM 個数を掛けて逆算する。

現在ライン（例：A工程）： 中間品X を生産
後ライン（例：B工程）： 完成品Y を生産（BOM: Y → X × 2個）

→ B工程 LineBacklog(Y, plan_date, plan_qty=100) から
→ A工程 LineBacklog(X, plan_date - LT, order_qty=200) に展開

LineBacklog の sequence_no

sequence_no	意味
-------------	----

sequence_no	意味
0	需要・在庫・実績などの基礎データ（自動削除なし）
1以上	ユーザー入力の生産計画（削除可能）

LineBacklog の主要フィールド

フィールド	内容
plan_date	生産予定日
order_qty	pickup で設定した需要数
demand_qty_plan	order_qty と同値（バックアップ）
plan_qty	ユーザー入力の計画数（初期値 0）
actual_qty	実績数（ガントチャート等から更新）
stock_qty	実在庫
planned_stock_qty	計画在庫
adjust_qty	手動調整数
scrap_qty	自工程仕損数
progress_qty	進度（顧客需要に対する実績の遅れ/先行）
sequence_no	0=基本、1+=計画

5. 進度計算

需要ソース

進度計算の需要は必ず **LineDemand** を使用する。LineBacklog の **order_qty** や **demand_qty_plan** をフォールバックに使ってはならない。

```
# progress_calculator.py
if demand.firm_qty and demand.firm_qty > 0:
    qty = demand.firm_qty # 確定優先
elif demand.forecast_qty and demand.forecast_qty > 0:
    qty = demand.forecast_qty # なければ内示
else:
    qty = 0
```

6. 業務ルール

日替わり時刻（8:00）

「今日」の判定は 8:00 を区切りとする。

- 8:00 未満 → 前日扱い
- 8:00 以上 → 当日扱い

連産品（is_coproduct=True）の除外

受注展開（BOM倍率計算・在庫計算）では連産品BOMを除外する。

```
BOM.objects.filter(is_coproduct=False, ...) # 展開対象から除外
```

棚卸初期化と通常計算の分離

区分	ファイル
通常計算（日次再計算）	inventory_calculator.py / progress_calculator.py
棚卸専用初期化	stocktake_initializer.py

棚卸専用の在庫・進捗再計算は stocktake_initializer.py 内の専用関数で行い、通常計算ファイルを流用しないこと。

7. 関連ファイル

機能	ファイル	クラス/関数
CSV取込・Order生成	apps/orders/core/services/csv_import.py	CSVImportService.create_orders_from_staging
受注展開サービス	apps/production/services/order_expansion.py	OrderExpansionService.expand_open_orders
BOM倍率計算	apps/production/services/order_expansion.py	_get_bom_multiplier_maps, _collect_bom_multipliers
LineDemand API	apps/production/views.py	LineDemandViewSet.expand
LineBacklog pickup	apps/production/views.py	LineBacklogViewSet.pickup
在庫計算	apps/production/inventory/inventory_calculator.py	aggregate_scrap_to_backlog
進捗計算	apps/production/inventory/progress_calculator.py	recalculate_progress_qty
棚卸初期化	apps/production/inventory/stocktake_initializer.py	（棚卸専用）

8. API エンドポイント

メソッド	エンドポイント	説明
POST	/api/line-demands/expand/	OPEN受注をLineDemandに展開（全件削除・再作成）
POST	/api/line-backlogs/pickup/	需要をLineBacklogに保存（最終品 + 中間品）
POST	/api/line-backlogs/pickup_purchase/	購買需要を集計